

Exercices on sessions 1 and 2 of Going further with R

Load the package `tidyverse`.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

Writing a function

We are going to write a function to standardise a variable to mean 0 and standard deviation 1. *There is already an existing function to do this (the function `scale`), so let's forget that this function exists.* Remember that to standardise a variable in this way, we need to do the following operation for each value x of the variable:

$$\frac{x - \text{mean}}{sd}$$

In addition to the variable to standardise, we want this function to take a second argument being the number of decimal we want the standardised values to be rounded to. We will give a default value of 2 to this argument.

- plan the function: identify the arguments and the steps

```
# Arguments
v # a vector of numerical value
decim <- 2 # the number of decimal to round the standardise values

# Body of the function
# calculate the mean
# calculate the sd
# calculate the standardised values
# round the standardised values
```

- create a test vector by randomly drawing 10 values in a normal distribution of mean 16 and standard deviation of 3

```
v_test <- rnorm(n=10, mean = 16, sd=3)
v_test
```

```
[1] 20.25045 18.27833 14.65139 20.59571 18.63919 17.78472 16.76210 20.27603
[9] 12.51493 14.55659
```

```
summary(v_test)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
12.51   15.18   18.03   17.43   19.85   20.60
```

- write the code of the function (for keeping thing simple, this function will always remove the NA)

```
# Arguments
v <- v_test # a vector of numerical value
decim <- 2 # the number of decimal to round the standardise values

# Body of the function
# calculate the mean
m_v <- mean(v, na.rm = TRUE)
# calculate the sd
sd_v <- sd(v, na.rm = TRUE)
# calculate the standardised values
v_sd <- (v - m_v) / sd_v
# round the standardised values
v_final <- round(v_sd, digits = decim)
```

- create the function

```
# Arguments
my_scale <- function(v, decim = 2) {
  # calculate the mean
  m_v <- mean(v, na.rm = TRUE)
  # calculate the sd
  sd_v <- sd(v, na.rm = TRUE)
  # calculate the standardised values
  v_sd <- (v - m_v) / sd_v
  # round the standardised values
  v_final <- round(v_sd, digits = decim)
  # return
  return(v_final)
}
```

- test the function

```
# remove test argument
rm(v, decim)

# test 1 - test the results
v_test1 <- rnorm(n=1000, mean = 25, sd=8)
res1 <- my_scale(v_test1)
mean(res1) # should be close to 0
```

```
[1] 3e-05
```

```
sd(res1) # should be close to 1
```

```
[1] 0.9999633
```

```
# test 2 - test if NA handle properly
v_test2 <-c(v_test1, NA)
res2 <- my_scale(v_test2)
table(c(res1, NA) == res2) # should be always true
```

```
TRUE
1000
```

```
# test 3 - change the number of decimals
head(res1)
```

```
[1] 0.20 0.50 1.46 0.42 0.22 -1.22
```

```
head(my_scale(v_test1, decim = 0))
```

```
[1] 0 1 1 0 0 -1
```

```
head(my_scale(v_test1, decim = 4))
```

```
[1] 0.2009 0.5003 1.4570 0.4185 0.2229 -1.2227
```

Conditional execution

- Write a function using conditional execution that take 2 arguments: the temperature (numerical) and the rain (boolean). This function will tell the user to:
 - go to the beach if the temperature is above 25°C and it's not raining
 - take their umbrella if the temperature is above 25°C and it's raining
 - take their jumper if the temperature is below 25°C and it's not raining
 - stay home otherwise

```
test_weather <- function(temp, rain) {
  if (temp >= 25 && rain == FALSE) {
    message("Go to the beach")
  } else if (temp > 25 && rain == TRUE) {
    message("Take your umbrella")
  } else if (temp < 25 && rain == FALSE) {
    message("Take your jumper")
  } else {
    message("Stay home")
  }
}
```

- Test this function

```
test_weather(temp = 32, rain = FALSE)
```

Go to the beach

```
test_weather(temp = 32, rain = TRUE)
```

Take your umbrella

```
test_weather(temp = 15, rain = FALSE)
```

Take your jumper

```
test_weather(temp = 15, rain = TRUE)
```

Stay home

- Add two tests at the beginning of function to check that the arguments are of the correct mode, and stop the execution of the function otherwise

```
test_weather <- function(temp, rain) {  
  # test the argument  
  if (!(is.numeric(temp)) | !(is.logical(rain))) {  
    stop("check the mode of your arguments")  
  }  
  
  # perform the test of wearther  
  if (temp > 25 & rain == FALSE) {  
    message("Go to the beach")  
  } else if (temp > 25 & rain == TRUE) {  
    message("Take your umbrella")  
  } else if (temp < 25 & rain == FALSE) {  
    message("Take your jumper")  
  } else {  
    message("Stay home")  
  }  
}  
  
# check
```

```

# test_weather(temp= 28, rain =12)
# test_weather(temp= "blue", rain = TRUE)

# Alternative
test_weather <- function(temp, rain) {
  # test the argument
  if (!(is.numeric(temp)) ) {
    stop("temp must be numeric")
  }
  if (!(is.logical(rain))) {
    stop("rain must be logical")
  }

  # perform the test of wearther
  if (temp > 25 & rain == FALSE) {
    message("Go to the beach")
  } else if (temp > 25 & rain == TRUE) {
    message("Take your umbrella")
  } else if (temp < 25 & rain == FALSE) {
    message("Take your jumper")
  } else {
    message("Stay home")
  }
}

# check
# test_weather(temp= 28, rain =12)
# test_weather(temp= "blue", rain = TRUE)

```

Loop

Write a `for` loop that calculates the cumulative sum of a vector. In other words: the first element of the output is the first element of the input, the second element of the output is the sum of the first two elements of the input, ... the n^{th} element of the output is the sum of the n^{th} first element of the input. *There is a function called `cumsum` that does this, but let's forget about it for now...* To run this loop, make a vector of 10 random draws between 0 and 4.

```

# create the input vector
input <- runif(n = 10, min = 0, max = 4)

```

```
# create an empty output to store the results
output <- vector("integer", length = length(input))

# initialise the cumulative sum
total <- 0

for (i in seq_along(input)) {
  total <- total + input[i]
  output[i] <- total
}

# for comparison with cumsum
output == cumsum(input)
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

Advanced tidyverse

- Create the following list and use one of the `map` functions from the package `purrr` to get the maximum value of `H` for each plot.

```
H_list <- list(plot1 = c(8, 25, 12, 16, 14),
               plot2 = c(31, 25, 18, 19, 29, 24, 16),
               plot3 = c(15, 12, 13, 17, 19, 11))
H_list
```

```
$plot1
[1] 8 25 12 16 14
```

```
$plot2
[1] 31 25 18 19 29 24 16
```

```
$plot3
[1] 15 12 13 17 19 11
```

```
map_dbl(H_list, max)
```

```
plot1 plot2 plot3
    25    31    19
```

```
# or
map_int(H_list, max)
```

```
plot1 plot2 plot3
    25    31    19
```

```
# or but less usefull
map(H_list, max)
```

```
$plot1
[1] 25
```

```
$plot2
[1] 31
```

```
$plot3
[1] 19
```

- Use the function **across** with the function you have created in the first section to standardise all the numerical variables of the data set iris

```
data("iris")
iris_sd <- iris %>% mutate(
  across(where(is.numeric),
    my_scale
  )
)
summary(iris_sd)
```

Sepal.Length	Sepal.Width	Petal.Length
Min. : -1.8600000	Min. : -2.4300000	Min. : -1.5600000
1st Qu.: -0.9000000	1st Qu.: -0.5900000	1st Qu.: -1.2200000
Median : -0.0500000	Median : -0.1300000	Median : 0.3350000
Mean : -0.0001333	Mean : 0.0008667	Mean : -0.0004667
3rd Qu.: 0.6700000	3rd Qu.: 0.5600000	3rd Qu.: 0.7600000
Max. : 2.4800000	Max. : 3.0800000	Max. : 1.7800000

Petal.Width	Species
Min. : -1.4400000	setosa : 50
1st Qu.: -1.1800000	versicolor: 50
Median : 0.1300000	virginica : 50

```
Mean    :-0.0003333
3rd Qu.: 0.7900000
Max.    : 1.7100000
```

```
# if we want to pass arguments to the function my_scale
iris_sd <- iris %>% mutate(
  across(where(is.numeric),
    my_scale,
    decim = 4
  )
)
```

```
Warning: There was 1 warning in `mutate()`.
i In argument: `across(where(is.numeric), my_scale, decim = 4)`.
Caused by warning:
! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
Supply arguments directly to `.fns` through an anonymous function instead.
```

```
# Previously
across(a:b, mean, na.rm = TRUE)
```

```
# Now
across(a:b, \(x) mean(x, na.rm = TRUE))
```

```
head(iris_sd)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	-0.8977	1.0156	-1.3358	-1.3111	setosa
2	-1.1392	-0.1315	-1.3358	-1.3111	setosa
3	-1.3807	0.3273	-1.3924	-1.3111	setosa
4	-1.5015	0.0979	-1.2791	-1.3111	setosa
5	-1.0184	1.2450	-1.3358	-1.3111	setosa
6	-0.5354	1.9333	-1.1658	-1.0487	setosa

```
# This work but is depreciated, so better to do it the alternative way, by creating an anonym
iris_sd <- iris %>% mutate(
  across(where(is.numeric),
    \(x) my_scale(x,decim = 4)
  )
)
head(iris_sd)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	-0.8977	1.0156	-1.3358	-1.3111	setosa
2	-1.1392	-0.1315	-1.3358	-1.3111	setosa
3	-1.3807	0.3273	-1.3924	-1.3111	setosa
4	-1.5015	0.0979	-1.2791	-1.3111	setosa
5	-1.0184	1.2450	-1.3358	-1.3111	setosa
6	-0.5354	1.9333	-1.1658	-1.0487	setosa

- Do the same thing but only for the variables related to sepals

```
iris_sd2 <- iris %>% mutate(
  across(starts_with("Sepal"),
    my_scale
  )
)
summary(iris_sd2)
```

Sepal.Length		Sepal.Width		Petal.Length		Petal.Width	
Min.	:-1.8600000	Min.	:-2.4300000	Min.	:1.000	Min.	:0.100
1st Qu.	:-0.9000000	1st Qu.	:-0.5900000	1st Qu.	:1.600	1st Qu.	:0.300
Median	:-0.0500000	Median	:-0.1300000	Median	:4.350	Median	:1.300
Mean	:-0.0001333	Mean	: 0.0008667	Mean	:3.758	Mean	:1.199
3rd Qu.	: 0.6700000	3rd Qu.	: 0.5600000	3rd Qu.	:5.100	3rd Qu.	:1.800
Max.	: 2.4800000	Max.	: 3.0800000	Max.	:6.900	Max.	:2.500

Species	
setosa	:50
versicolor	:50
virginica	:50